

Wie funktioniert ChatGPT wirklich?

Von einfachen Bausteinen zu künstlicher Intelligenz

Embeddings – Recap

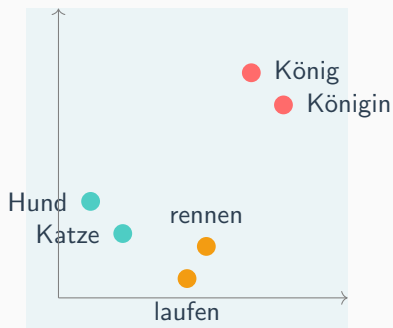
Wörter sind Punkte im Raum

Wie gerade gesehen:

- Jedes Wort hat einen **Ort** in einem hochdimensionalen Raum
- Ähnliche Bedeutung = nahe beieinander
- Diese Punkte sind der **Startpunkt** für alles Weitere

Die Frage jetzt:

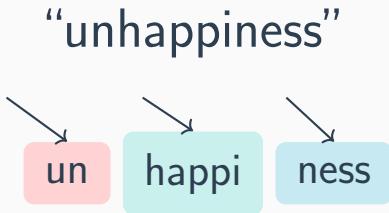
Was macht die KI mit diesen Punkten?



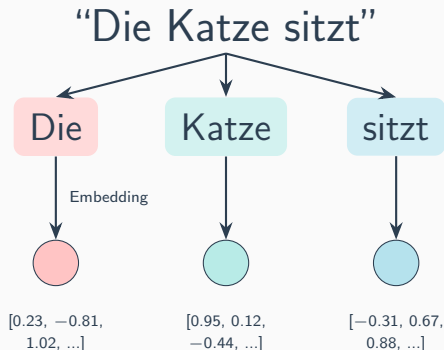
Tokenization

Wie liest die KI Text?

Die KI liest weder in Buchstaben noch in Wörtern – sondern in **Tokens**: häufige Silben und Wortteile.



Jedes Token wird zum Punkt



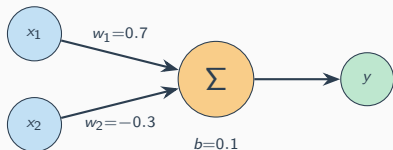
Jedes Token ist jetzt ein Punkt mit **tausenden Koordinaten**.
Diese Punkte werden jetzt **verarbeitet**.

Dense Layer und ReLU

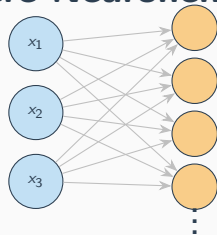
Baustein 1: Das Dense Layer

Ein einzelnes Neuron:

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$



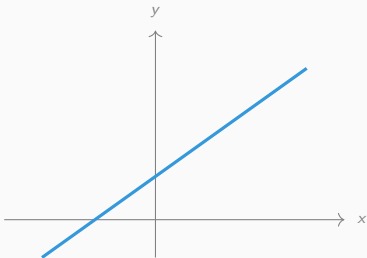
Mehrere Neuronen = ein



Layer:

bis zu Millionen!

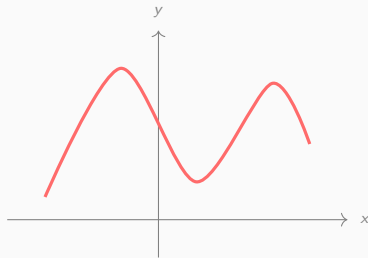
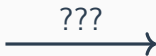
Das Problem: Linear kann nur Geraden



Nur linear: Gerade

✓ einfach

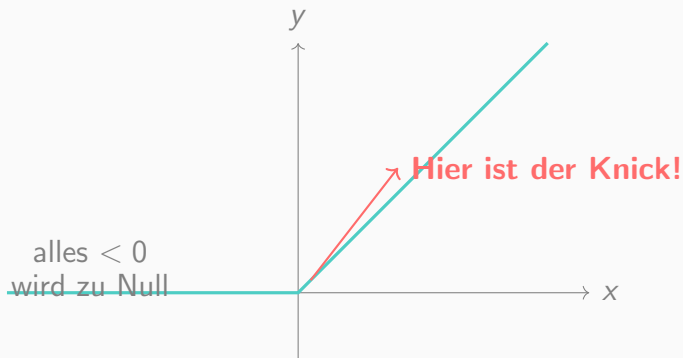
✗ langweilig



Realität: Komplex!

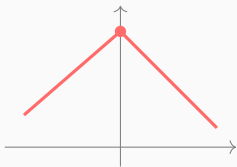
Wie kriegen wir das hin?

Die Lösung: ReLU macht Knicke!

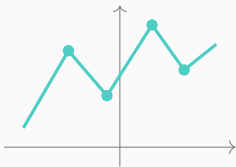


$$\text{ReLU}(x) = \max(0, x)$$

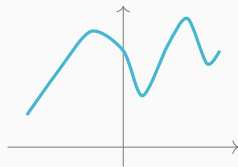
Mehr Neuronen = Mehr Knicke = Jede Kurve



2 Neuronen



5 Neuronen



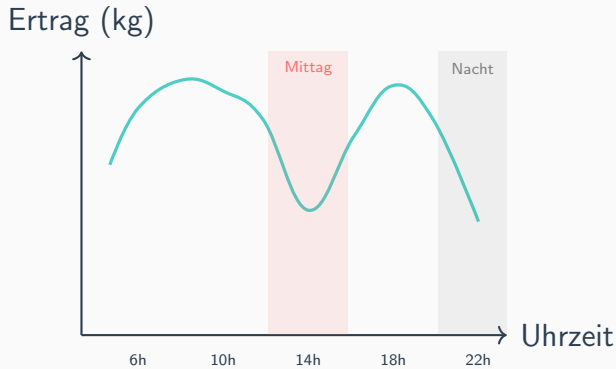
Viele Neuronen
 $\approx$ beliebige Kurve!

Universal Approximation Theorem:

Dense + ReLU + Dense kann *jede* Funktion approximieren.

Mehr Neuronen = genauer.

Beispiel: Bewässerung und Ernte



Das Netzwerk lernt:

- Mittags wässern → schlecht
- Nachts wässern → schlecht
- Morgens/Abends → gut

Aber es weiß nicht:

- Mittags: Tropfen = Linse → Sonnenbrand
- Nachts: Feuchtigkeit →

Der Transformer

Jetzt: Wie wird daraus eine KI?

Wir haben jetzt zwei Bausteine:

Dense + ReLU

Kann alles berechnen

+

Attention

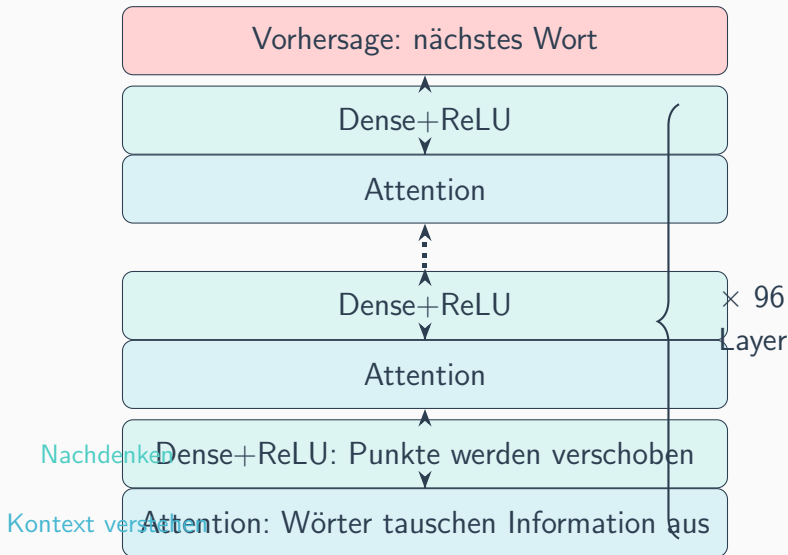
Verbindet Wörter

= Ein Transformer-Layer

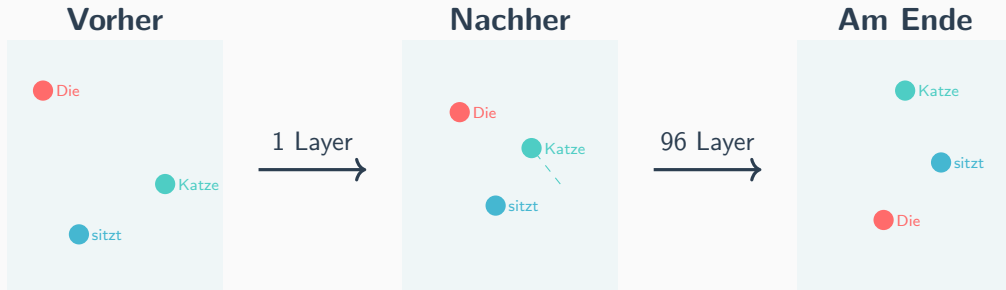
Und davon stapelt man 96 Stück übereinander.

(GPT-4: ca. 120 Layer, 1.8 Billionen Parameter)

Ein Layer nach dem anderen



Was macht ein Layer mit den Punkten?

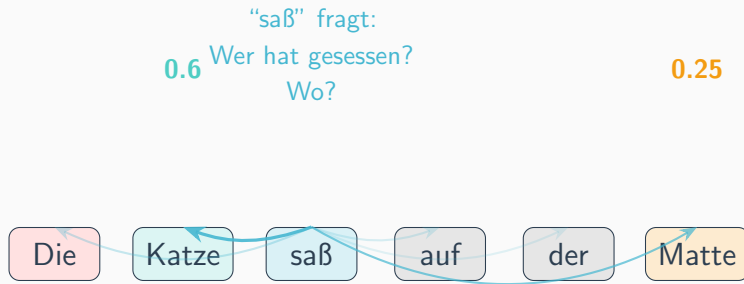


Jeder Layer verschiebt die Punkte ein Stück.

Am Ende kodiert “Katze” nicht mehr nur *Katze*,
sondern **“die Katze die hier sitzt und über die wir reden”**.

Attention im Detail

Attention: Jedes Wort schaut alle anderen an



Jedes Wort stellt eine “Frage” an alle anderen.
Die Antworten bestimmen, wohin der Punkt verschoben wird.

Attention: Query, Key, Value

Jedes Token berechnet drei Dinge:

Query (Q)

Was suche ich?

Key (K)

Was bin ich?

Value (V)

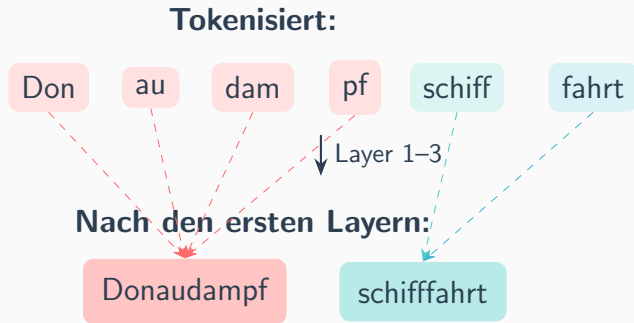
Mein Inhalt

Query von "saß" · Key von "Katze" = **hoher Wert!**

→ Also fließt der Value von "Katze" in "saß" ein.

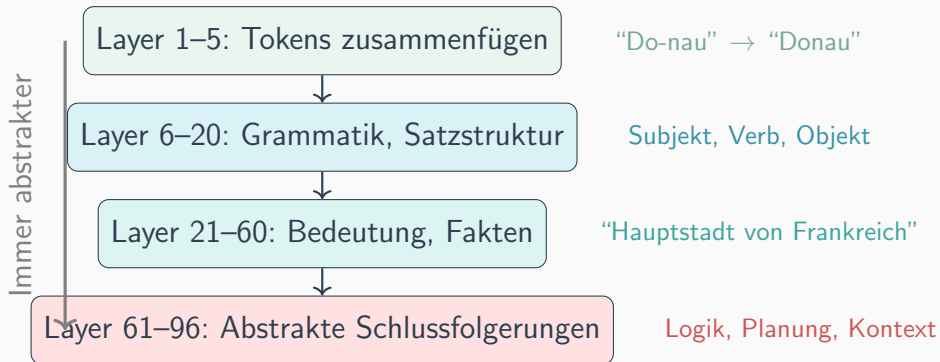
$$\text{Attention} = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$

Frühe Layer: Tokens zusammenfügen



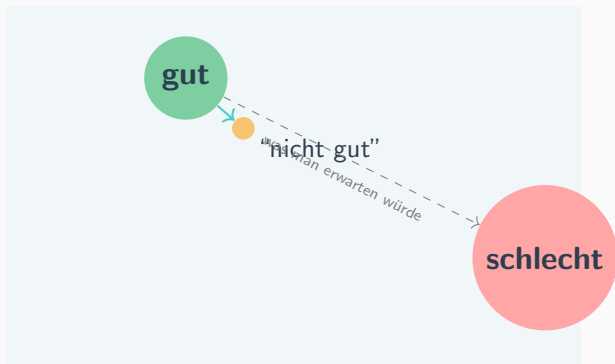
Attention in frühen Layern erkennt:
“Diese Tokens gehören zusammen!”

Spätere Layer: Immer abstrakter



Die späten Layer sind am schwersten zu verstehen – dort passiert das "Denken", aber niemand weiß genau wie.

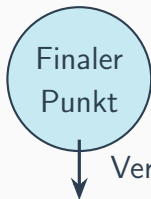
Nebeneffekt: Warum “nicht gut” $\neq$ “schlecht”



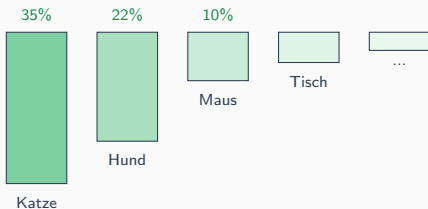
- Attention bildet **gewichtete Summen** – es mittelt zwischen Punkten
- “nicht” verschiebt “gut” nur ein bisschen weg – nicht bis zu

Next-Token-Prediction

Am Ende: Welches Wort kommt als nächstes?



Skalarprodukt $\rightarrow$ Ähnlichkeit $\rightarrow$ Wahrscheinlichkeit



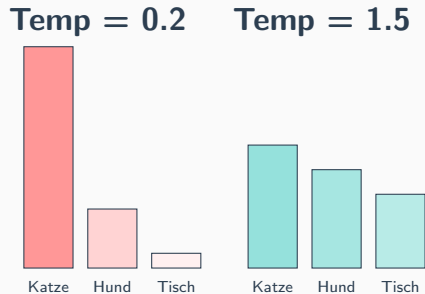
Sampling: Zufall macht kreativ

Temperature:

- Niedrig (0.1): Fast immer das wahrscheinlichste Wort
- Hoch (1.5): Auch unwahrscheinliche Wörter haben eine Chance

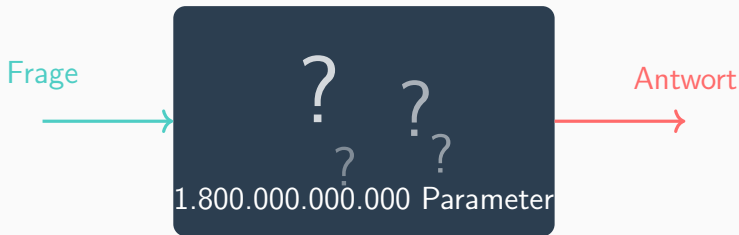
Top-K Sampling:

- Nur die besten K Kandidaten werden betrachtet
- Rest wird ignoriert



Die Black Box

Das große Geheimnis



Wir kennen die Architektur.

Wir kennen die Gewichte.

Aber wir verstehen nicht, was sie bedeuten.

Mechanistic Interpretability: Die Box öffnen

Ein junges Forschungsfeld versucht zu verstehen, **welche Algorithmen** das Netzwerk intern gelernt hat.

Die Idee:

- Einzelne Neuronen beobachten
- “Schaltkreise” finden
- Verstehen was *wo* gespeichert ist

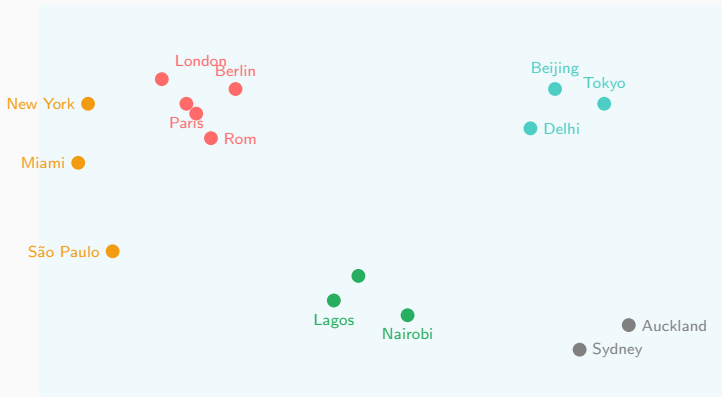
Bisherige Funde:

- Induction Heads (Muster-Erkennung)
- Fakten in bestimmten Layern
- Mathematische Algorithmen
- Geographische Karten!

Faszinierende Entdeckungen

Entdeckung 1: Die Weltkarte

Städte-Embeddings aus einem LLM:



Das Modell hat **nie eine Karte gesehen** – nur Text!

Entdeckung 2: Modular Addition

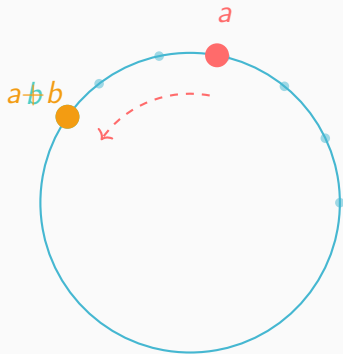
Aufgabe:

Trainiere ein kleines Netzwerk
auf:

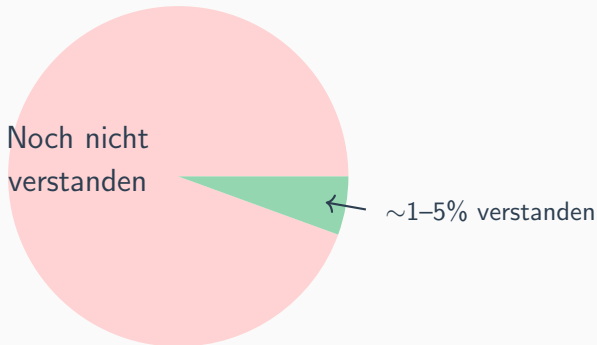
$$(a + b) \mod 113$$

Was es intern lernt:

1. Zahlen als Punkte auf einem **Kreis** darstellen
2. Addition = **Rotation**



Was wir wissen – und was nicht



Offene Fragen:

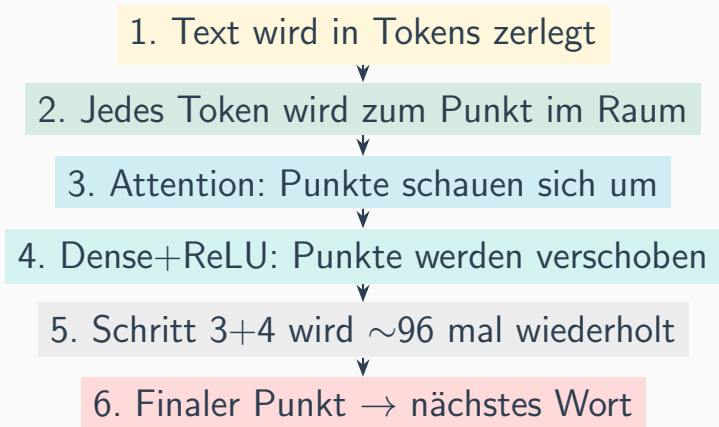
- Haben LLMs ein "Weltmodell" ?

Theorien:

- Kompression des Internets
- Simulation von "Autoren"

Zusammenfassung

Der komplette Weg – auf einen Blick



Was wir heute gelernt haben

1. **Tokens:** KI liest in Silben-Stücken, nicht in Wörtern
2. **Dense + ReLU:** Kann mit genug Neuronen *alles* berechnen
3. **Attention:** Lässt Wörter miteinander kommunizieren
4. **Layer:** Frühe = einfach, späte = abstrakt
5. **Vorhersage:** Am Ende wird das wahrscheinlichste nächste Wort gewählt
6. **Black Box:** Wir verstehen noch fast nichts vom Innenleben
7. **Aber:** Faszinierende Strukturen emergieren (Weltkarte, Fourier...)

Danke!

Fragen?

Quellen zum Vertiefen:

Weltkarte:	Gurnee & Tegmark (2023), arXiv:2310.02207
Mod 113:	Neel Nanda et al. (2023), arXiv:2301.05217
Attention:	Vaswani et al. (2017), "Attention Is All You Need"